

ANÁLISE COMPARATIVA ENTRE THREADS TRADICIONAIS E VIRTUAIS NO JAVA

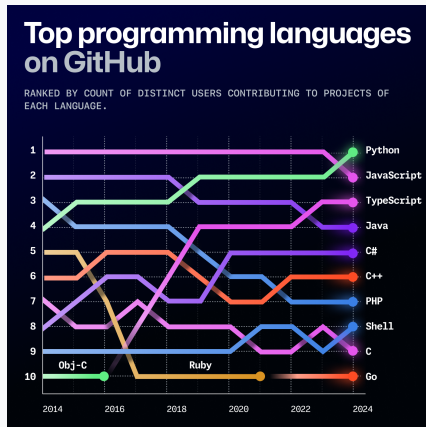
Stephanye Cristine Antunes De Cunto

11 de janeiro de 2026

Introdução

Java é uma das linguagens de programação mais utilizadas no mundo, ocupando posição de destaque entre as linguagens mais populares no GitHub.

- ▶ Linguagem multiplataforma
- ▶ Forte presença no mercado
- ▶ Muito utilizada em sistemas concorrentes



Fonte: GitHub — *State of the Octoverse 2024*

Java no Cenário Atual

Popularidade

- ▶ Top 5 no GitHub
- ▶ Grande comunidade
- ▶ Ecossistema consolidado

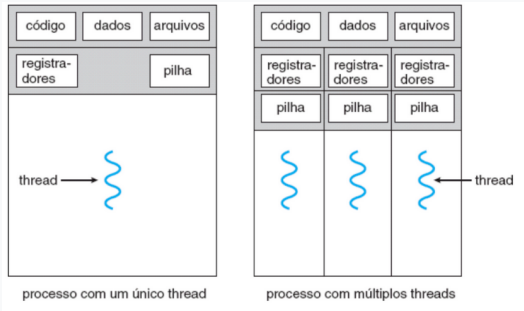
Principais Usos

- ▶ Sistemas corporativos
- ▶ Servidores e APIs
- ▶ Aplicações de alto desempenho

O que são Threads?

Threads são unidades de execução independentes dentro de um mesmo processo, permitindo que múltiplas tarefas sejam executadas simultaneamente.

- ▶ Compartilham memória
- ▶ Executam em paralelo
- ▶ Melhoram a eficiência do sistema



Fonte: Silberschatz, Galvin e Gagne (2018).

Por que usar Threads?

A utilização de threads possibilita o aproveitamento eficiente de processadores multicore, aumentando o desempenho da aplicação.

- ▶ Execução concorrente
- ▶ Redução de tempo de espera
- ▶ Maior responsividade

Benefícios das Threads

Desempenho

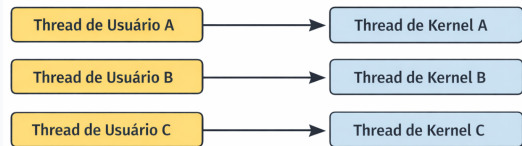
- ▶ Uso eficiente da CPU
- ▶ Execução paralela

Escalabilidade

- ▶ Suporte a múltiplas requisições
- ▶ Melhor resposta do sistema

Threads tradicionais em Java

- ▶ Modelo de mapeamento *one-to-one* (1:1)
- ▶ Alto consumo de recursos
- ▶ Criação de threads é custosa
- ▶ Número de threads limitado pelo SO



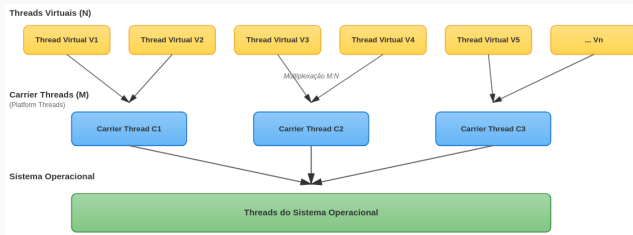
Fonte: Elaborado pela própria autora.

Limitações das Threads Tradicionais

- ▶ Alto consumo de memória (1 MB de pilha por thread)
- ▶ Criação custosa (system calls e kernel)
- ▶ Overhead elevado de troca de contexto
- ▶ Baixa escalabilidade
- ▶ Problema C10K (+10.000 conexões simultâneas)
- ▶ Competição por recursos do sistema

Threads Virtuais

- ▶ Executadas sobre um conjunto reduzido de *platform threads* (*carrier threads*)
- ▶ Multiplexação de múltiplas threads virtuais
- ▶ Modelo análogo a M:N (N threads lógicas para M físicas)
- ▶ Melhor escalabilidade do sistema
- ▶ Baixo consumo de recursos



Fonte: Elaborado pela própria autora.

Threads Virtuais: Mounting e Unmounting

Conceitos Fundamentais:

- ▶ **Mounting:** thread virtual é associada a uma carrier thread para execução
- ▶ **Unmounting:** thread virtual é liberada da carrier thread

Funcionamento:

1. Thread virtual executa normalmente (mounted)
2. Ao encontrar operação bloqueante (I/O, sleep, wait), sofre *unmounting*
3. Carrier thread fica livre para executar outras threads virtuais
4. Após conclusão da operação, ocorre *remounting* em carrier disponível

Benefícios e Casos de Uso

Vantagens Principais:

1. **Escalabilidade:** Suporte a milhares de threads concorrentes
2. **Eficiência de Memória:** Pilha dinâmica iniciando em ~ 1 KB
3. **Menor Overhead:** Redução significativa no consumo de recursos

Caso de Uso Ideal

Threads virtuais são **adequadas para workloads I/O-bound**, onde o tempo é gasto aguardando operações de entrada/saída.

Limitação

Para tarefas **CPU-bound**, o benefício é limitado, o gargalo permanece sendo o número de cores disponíveis.

Metodologia

Base do Experimento: Replicação do trabalho de Gustafsson (2024)

Servidor

- ▶ Spring Boot HTTP
- ▶ Endpoints: thread tradicional, thread virtual, GC
- ▶ MacBook Air M2
- ▶ 8 GB RAM, macOS

Cliente

- ▶ Gerador de carga
- ▶ Intel i5-2410M
- ▶ 8 GB RAM, Ubuntu

Infraestrutura: Conexão Wi-Fi com ajustes de sistema para otimização

Cenário 1: Carga Constante

Objetivo: Avaliar a estabilidade e o desempenho dos mecanismos de threads sob carga constante ao longo do tempo

Parâmetros do Teste:

- ▶ **Carga:** 1.000 requisições por segundo
- ▶ **Duração:** 10 minutos por execução
- ▶ **Repetições:** 50 execuções
- ▶ **Configuração:** Parâmetros baseados no trabalho de Gustafsson, exceto a carga aplicada

Cenário 2: Aumento Gradual de Carga (Ramp-Up)

Objetivo: Identificar o ponto de saturação onde o servidor começa a apresentar erros de conexão

Procedimento do Teste:

1. Envio de requisições durante 10 segundos
2. Pausa e execução de Garbage Collection
3. Incremento de 50 requisições/segundo
4. Repetição do ciclo até ocorrerem 3 erros consecutivos

Parâmetros:

- ▶ **Repetições:** 50 execuções
- ▶ **Monitoramento:** Script automatizado para detecção de erros
- ▶ **Configuração:** Parâmetros baseados no trabalho de Gustafsson

Limitações dos Testes

1. Limitação do Sistema Operacional

- ▶ Restrições do SO impediram atingir limites absolutos de hardware
- ▶ Resultados refletem capacidade do SO, não do hardware

2. Infraestrutura de Rede

- ▶ Wi-Fi pode ter introduzido variabilidade de latência
- ▶ Rede cabeada poderia revelar diferenças mais sutis

3. Carga Sintética

- ▶ Ambiente controlado vs. produção com padrões variados

4. Simplificação da Aplicação

- ▶ Uso apenas de `sleep`, sem I/O real (BD, APIs)
- ▶ Limita generalização para aplicações reais

Métricas Coletadas

Servidor

- ▶ *Ferramentas:* Java Flight Recorder (JFR) e Java Mission Control (JMC)
- ▶ Utilização média de CPU (%)
- ▶ Consumo de memória RAM (MB)
- ▶ Utilização de heap da JVM (MB)

Cliente

- ▶ *Ferramenta:* Vegeta
- ▶ Latência (mín., máx., média, P50, P90, P95, P99)
- ▶ Throughput (req/s)
- ▶ Taxa de sucesso (%)
- ▶ Volume de dados (bytes)

Tratamento dos Dados:

- ▶ Remoção de outliers pelo método dos quartis (CPU, memória, heap)
- ▶ **Cenário 1:** Análise da latência média
- ▶ **Cenário 2:** Análise do throughput máximo (rate máximo)

Resultados: Cenário 1 - Carga Constante

Tabela: Média da latência sob carga constante

Endpoint	Lat. Média (s)	P90 (s)	P95 (s)	P99 (s)	Desvio padrão (s)
Virtual	0,1438	0,1398	0,2189	1,0600	0,4370
Tradicional	0,1762	0,1553	0,3346	1,9432	0,6523

Fonte: Elaborado pela própria autora.

Conclusão: Threads virtuais superam threads tradicionais em todas as métricas de latência, com melhor desempenho e maior previsibilidade.

Resultados: Cenário 1 - Utilização de Recursos

Tabela: Utilização média de recursos sob carga constante

Endpoint	CPU (%)	Memória (MB)	Heap (MB)
Virtual	3,70 $\pm$ 0,11	410,72 $\pm$ 149,33	142,24 $\pm$ 86,77
Tradicional	8,26 $\pm$ 0,10	462,22 $\pm$ 124,54	179,63 $\pm$ 73,73
Redução	55,2%	11,1%	20,8%

Fonte: Elaborado pela própria autora.

Conclusão: Threads virtuais são significativamente mais eficientes no uso de CPU (55% menos) e apresentam menor consumo de memória e heap. O maior desvio padrão na memória reflete o gerenciamento dinâmico de pilhas pela JVM.

Resultados: Cenário 2 - Carga Gradual (Ramp-Up)

Tabela: Taxa média de requisições (rate) sob carga gradual

Endpoint	Rate (req/s)	Desvio Padrão
Virtual	2.072,14	828,42
Tradicional	2.008,16	940,78

Fonte: Elaborado pela própria autora.

Principais Observações:

- ▶ Desempenho similar: diferença de apenas 3,2% (não estatisticamente significativa)
- ▶ Threads virtuais apresentam menor variabilidade (desvio 11,9% menor)
- ▶ Elevados desvios esperados devido à natureza progressiva do teste

Resultados: Cenário 2 - Utilização de Recursos

Tabela: Utilização média de recursos sob carga gradual

Endpoint	CPU (%)	Memória (MB)	Heap (MB)
Virtual	2,26 $\pm$ 0,73	478,40 $\pm$ 0,07	188,96 $\pm$ 43,32
Tradicional	3,95 $\pm$ 1,54	479,08 $\pm$ 0,10	195,34 $\pm$ 56,36

Fonte: Elaborado pela própria autora.

Principais Observações:

- ▶ CPU 42,8% menor para threads virtuais, com maior estabilidade (menor desvio)
- ▶ Consumo de memória e heap praticamente equivalentes
- ▶ Próximo à saturação, gargalos do sistema (limites de conexão) mascaram benefícios das threads virtuais

Conclusões dos Testes

Cenário 1 - Carga Constante (1.000 req/s):

- ▶ Threads virtuais superiores em todas as métricas
- ▶ Latência 18% menor, CPU 55% menor
- ▶ Maior estabilidade e previsibilidade

Cenário 2 - Carga Gradual (Ramp-Up):

- ▶ Throughput máximo similar (diferença de 3,2%)
- ▶ Consumo de recursos converge próximo à saturação
- ▶ Threads virtuais mantêm menor CPU e maior estabilidade

Causa da Convergência: Gargalos do sistema (limites de conexão do SO) mascaram os benefícios das threads virtuais em cargas extremas.

Implicações Práticas

Quando Usar Threads Virtuais?

Cargas Normais a Moderadas

Recomendação: Threads virtuais são preferíveis

- ▶ Menor latência e uso de CPU
- ▶ Maior estabilidade e previsibilidade
- ▶ Ideal para a maioria dos cenários de produção

Cenários de Carga Extrema

Observação: Desempenho converge devido a gargalos externos

- ▶ Limitações do SO tornam-se predominantes
- ▶ Threads virtuais ainda mantêm vantagem em estabilidade
- ▶ Benefícios do modelo parcialmente mascarados

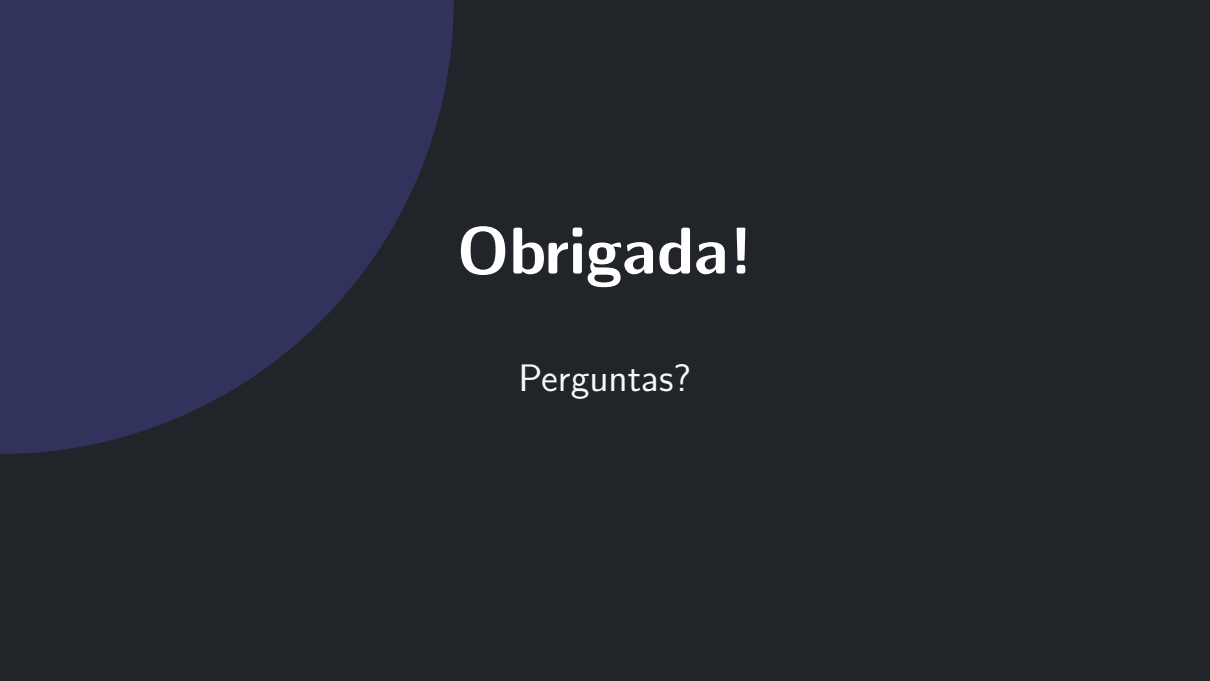
Conclusão Final

Resultados Principais:

- ▶ **Carga Constante:** Threads virtuais superiores em latência, CPU, memória e estabilidade
- ▶ **Carga Gradual:** Vantagens menos expressivas — throughput similar, consumo de recursos converge
- ▶ Limitações do SO mascaram benefícios em cenários extremos

Conclusão Geral:

- ▶ Threads virtuais sempre apresentam menor uso de CPU
- ▶ Consumo de memória e heap igual ou inferior
- ▶ Benefícios mais evidentes sob cargas normais/moderadas
- ▶ Adequadas para aplicações que demandam escalabilidade e previsibilidade



Obrigada!

Perguntas?